



StepForge

Free & open source · MIT

Record once. Test every layer. Export anywhere.

A local QA studio for UI, API, database, email and performance testing — with failures it explains itself. A case study by **Md Zarin Tasnim**.



THE PROBLEM

Testing one feature means juggling a different tool for every layer.

A real user journey crosses the UI, the API, the database and the inbox. The tools that test those layers don't talk to each other — and when a test goes red, nobody knows why.



Scattered tools

A recorder for the UI, a client for the API, a SQL tool for the data, a mail catcher, a load tester — each with its own login and no shared variables.



Red, but why?

A failed run says `TimeoutError`. Someone still has to dig out whether it's the app, the test, the data or the environment.



Brittle scripts

Recorded scripts lock onto one CSS path. A class name changes and half the suite turns red overnight.



Data in the cloud

Cloud test platforms want your credentials, screenshots and test data on their servers.



Paid per seat

The all-in-one options cost money per user, per month, per parallel run.



Locked in

Tests built in a studio stay in the studio. Moving to code later means starting over.

THE SOLUTION

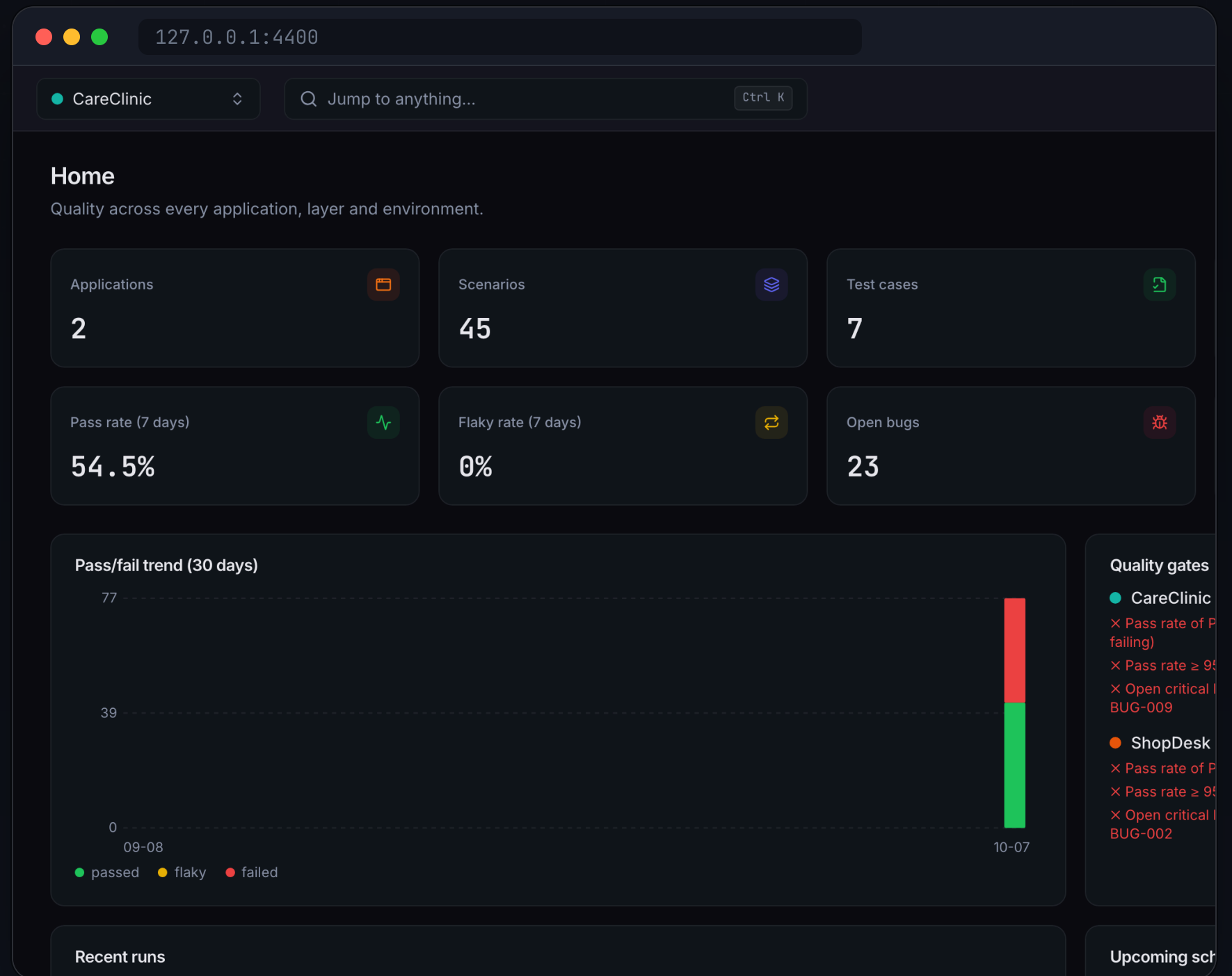
One local studio for every layer — and it tells you why a test failed.

- Record a journey in a real browser, with smart locators and fallbacks.
- Mix layers in one scenario: UI, API, SQL, email, load.
- Run in parallel on Chromium, Firefox and WebKit, desktop or phone.
- Diagnose every failure: where, why, whose issue, how to fix.
- Export clean code to the framework your team already uses.

127.0.0.1 only

No cloud

No paid services



RECORD ONCE

Click through your app. Get a readable, resilient test.

Start in the Recorder, or right from a scenario in the Test Explorer. A toolbar inside the browser adds checks, extracts values and masks passwords; every step keeps several locators, so a renamed class doesn't break it.

careclinic · recording

CareClinicDashboardPatientsAppointmentsSigned in as Riley Reception (receptionist) · Sign out

Appointment booked

Appointments

Book appointment

DATE	TIME	PATIENT	DOCTOR	REASON
2026-11-02	09:00	Ana Lopez (PAT-1001)	Dr. Dana Reyes	Annual check-up
2026-11-02	10:30	Ben Carter (PAT-1002)	Dr. Omar Haddad	Chest pain follow-up
2026-12-21	09:00	Ben Carter (PAT-1002)	Dr. Omar Haddad	Follow-up
2027-06-16	09:30	Ana Lopez (PAT-1001)	Dr. Omar Haddad	Check-up
2028-11-29	15:00	Hershel DuBuque (PAT-1008)	Dr. Mei Lin	Follow-up
2029-06-13	14:30	Chloe Nguyen (PAT-1003)	Dr. Mei Lin	Vaccination
2029-08-24	11:00	Ana Lopez (PAT-1001)	Dr. Omar Haddad	
2029-08-24	11:00	Ben Carter (PAT-1002)	Dr. Omar Haddad	
2030-03-13	10:00	Ana Lopez (PAT-1001)	Dr. Dana Reyes	Check-up
2031-11-23	09:30	Ana Lopez (PAT-1001)	Dr. Omar Haddad	Check-up
2032-02-22	10:00	Ana Lopez (PAT-1001)	Dr. Dana Reyes	Check-up
2032-12-14	14:00			

PauseAssertExtractMaskInsert...Undo13 stepsStop

StepForge · Recorder

Recordinghttp://127.0.0.1:52101/LoginAdding to "Receptionist books a follow-up"UndoDiscardStop & review

1Open /login

2Type "reception@careclinic.test" into the "Email" field

role

3Type "{{secret.password}}" into the "Password" field

testId

4Click the "Sign in" button

testId

5Check that "Riley Reception" is shown

css

6Click the "Appointments" link

testId

7Click the "Book appointment" link

testId

8Select "Ben Carter (PAT-1002)" in the "Patient" dropdown

testId

9Select "Dr. Omar Haddad · Cardiology" in the "Doctor" dropdown

role

10Type "2026-12-21" into the "Date" field

testId

11Type "Follow-up" into the "Reason for visit" field

role

12Click the "Book appointment" button

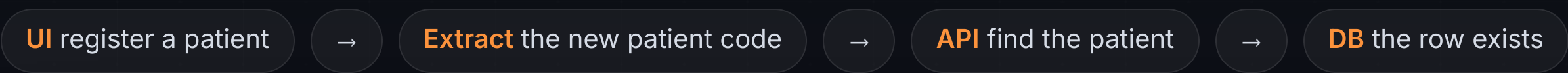
testId

13Check that the "Appointment booked" status message reads "Appointment booked"

testId

EVERY LAYER

One scenario: fill the form, call the API, check the row.



The screenshot displays the StepForge Explorer interface. On the left, a sidebar shows a tree view of modules and test cases under the 'CareClinic' project. The main area shows a detailed view of a test scenario titled 'Register in the UI, check the API and the database'. The scenario is marked as 'ready' and 'P1'. It consists of 11 steps, with the first 9 steps visible in the list view. The steps are:

1. Sign in (util.useBlock)
2. Open the registration form (ui.click)
3. A patient name (util.generateData -> vars.patient)
4. Full name (ui.fill)
5. Date of birth (ui.fill)
6. Phone (ui.fill)
7. Save (ui.click)
8. Read the new patient code (ui.extract -> vars.code)
9. API login (util.useBlock)

A built-in API client and SQL workbench, safe by default.

Import OpenAPI or Postman, check every response against the contract, and query five database engines. Connections start read-only, with every write rolled back.

The image shows two overlapping windows from the StepForge application. The background window is the 'API Client' and the foreground window is the 'SQL Workbench'.

StepForge · API Client

- Method: **POST**
- URL: `{{env.baseUrl}}/api/auth/login`
- Environment: **Local**
- Body (JSON):

```
{"email": "admin@careclinic.test", "password": "{{secret.adminPassword}}"}"
```
- Status: **200 OK** (2 ms, 143 B)
- Response Body:

```
{  "token": "17c7cf6e52dae46842fb8de932c58399a82ebb6826d95527",  "user": {    "id": 1,
```

StepForge · SQL Workbench

- Connection: **clinic · Local (SQLite)**
- Mode: **read-only** (rollback mode)
- Schema:
 - Filter 6 tables...
 - appointments (8)
 - doctors (4)
 - email_codes (3)
 - patients (8)
 - sessions (3)
 - users (6)
- Query:

```
1 SELECT d.name AS doctor, d.specialty, COUNT(a.id) AS appointments
2 FROM doctors d LEFT JOIN appointments a ON a.doctor_id = d.id
3 GROUP BY d.id ORDER BY appointments DESC;
```
- Run: **Run** (Ctrl + Enter)
- Result: 3 rows in 1 ms
- Table View:

#	doctor	specialty
1	Dr. Omar Haddad	Cardiology
2	Dr. Dana Reyes	General Practice
3	Dr. Malika	Pediatrics

Sign-up flows tested end to end, inbox included.

- A **local mail catcher** (Mailpit) receives every message — nothing is sent out.
- Steps wait for the email, **check the subject and body**, and pull out the **one-time code** or the link.
- The code goes straight into the next UI step.
- A broken verification link fails the test and is reported as a bug.

The screenshot displays the 'StepForge · Run results' window. At the top, it indicates '1 worker · started 1 min ago · 33.8 s'. A progress bar shows 28/28 steps completed. The main section lists the steps of the 'Sign up with email verification' scenario (Email · v1):

Step	Action	Details	Duration	Thumbnail
1	🔗	A unique address for this run	vars.email set	0 ms
2	🖥️	Open the sign-up page	Opened http://127.0.0.1:52101/signup (200)	80 ms
3	🖥️	Full name		34 ms
4	🖥️	Email		15 ms
5	🖥️	Password		51 ms
6	🖥️	Create the account		79 ms
7	✉️	Wait for the verification email	Received "Verify your CareClinic account" from no-reply@careclinic.test after 0.0 s	3 ms
8	✉️	It has a code and a link	Checked "Verify your CareClinic account"	1 ms
9	✉️	Read the code	Extracted code 816900	0 ms
10	🖥️	Enter the code		29 ms
11	🖥️	Verify		74 ms
12	🖥️	Verified	Assert urlContains "verified=1"	15 ms

Load tests with thresholds, live charts and a k6 export.



- Smoke, load, stress, spike and soak profiles.
- p95, p99, error rate and throughput thresholds that pass or fail a run.
- Built-in engine, or k6 when installed.
- Response-time checks and Lighthouse for pages.
- Asks you to confirm you own the system before any load test.

Every red test comes with an explanation and a bug report.

A rule engine reads the failed step, the network log, console errors and the DOM, then says where it broke, why, whose issue it is and how to fix it — no AI required.

The screenshot displays the StepForge 'Run results' window. At the top, the 'CareClinic' project is selected, and a search bar is available. A progress bar indicates 16 passed, 12 failed, 0 broken, 0 flaky, and 0 skipped tests out of 28/28. Below this, a list of test results is shown, with the 'Insured patients get 20% off' test highlighted as failed. The detailed view for this failed test shows the error message: 'Expected \$.discount equals 12, but got 0; Expected \$.total equals 48, but got 60'. The diagnosis identifies this as an 'Application bug' (assertion mismatch) with 60% confidence, occurring in Step 4 — Get the bill. The explanation states: 'The response has 0 where the test expects 12. Expected \$.discount equals 12, but got 0 (response 200 from GET http://127.0.0.1:52101/api/appointments/15/bill)'. The 'Whose issue' is identified as an 'Application bug', and the 'How to fix' is: 'If the API is wrong, fix it; if the expectation changed, update the assertion.'.

StepForge · Run results

CareClinic

Jump to anything... Ctrl K

Local Run

← Runs

Failed CareClinic #M76VBFZZ

Local · http://127.0.0.1:52101 · chromium · desktop · 1 worker · started 1 min ago · 33.8 s

Run again Export

16 passed 12 failed 0 broken 0 flaky 0 skipped 28/28

All Failed Passed Flaky Skipped

✓ A wrong password is refused · TC... 1.1 s
Sign in

✓ A wrong password is refused · T... 382 ms
Sign in

✓ Sign in with valid credentials 382 ms
Sign in

✗ Dashboard labels the appointment... 6.0 s
Dashboard

✗ Dashboard lists the next appointm... 5.9 s
Dashboard

✓ Register a patient 622 ms
Patients

✗ Insured patients get 20% off
Billing · v1

Open scenario

Expected \$.discount equals 12, but got 0; Expected \$.total equals 48, but got 60

Application bug assertion mismatch confidence 60% BUG-005 · seen 2×

Where Step 4 — Get the bill

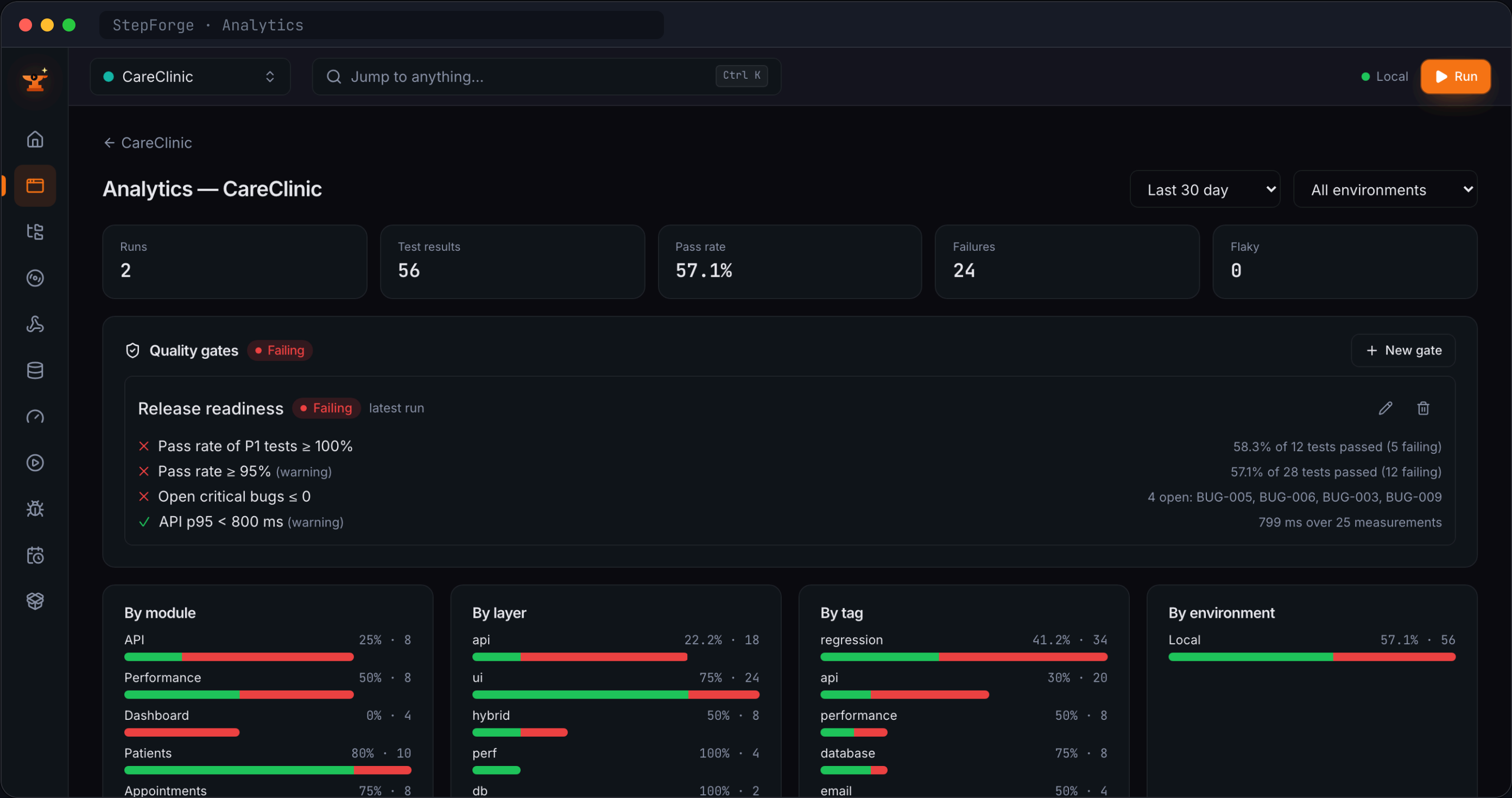
Why (likely) The response has 0 where the test expects 12
Expected \$.discount equals 12, but got 0 (response 200 from GET
http://127.0.0.1:52101/api/appointments/15/bill).

Whose issue Application bug

How to fix If the API is wrong, fix it; if the expectation changed, update the assertion.

Know if you can ship, at a glance.

Pass rates by module, layer, tag and environment; flaky-test detection; run comparison; and release gates that turn red when the rules aren't met.



EXPORT ANYWHERE

No lock-in: export clean, runnable code.

Playwright · TS

Playwright · Python

Cypress

Selenium · Python

Selenium · Java

pytest + requests

REST Assured

k6

Postman

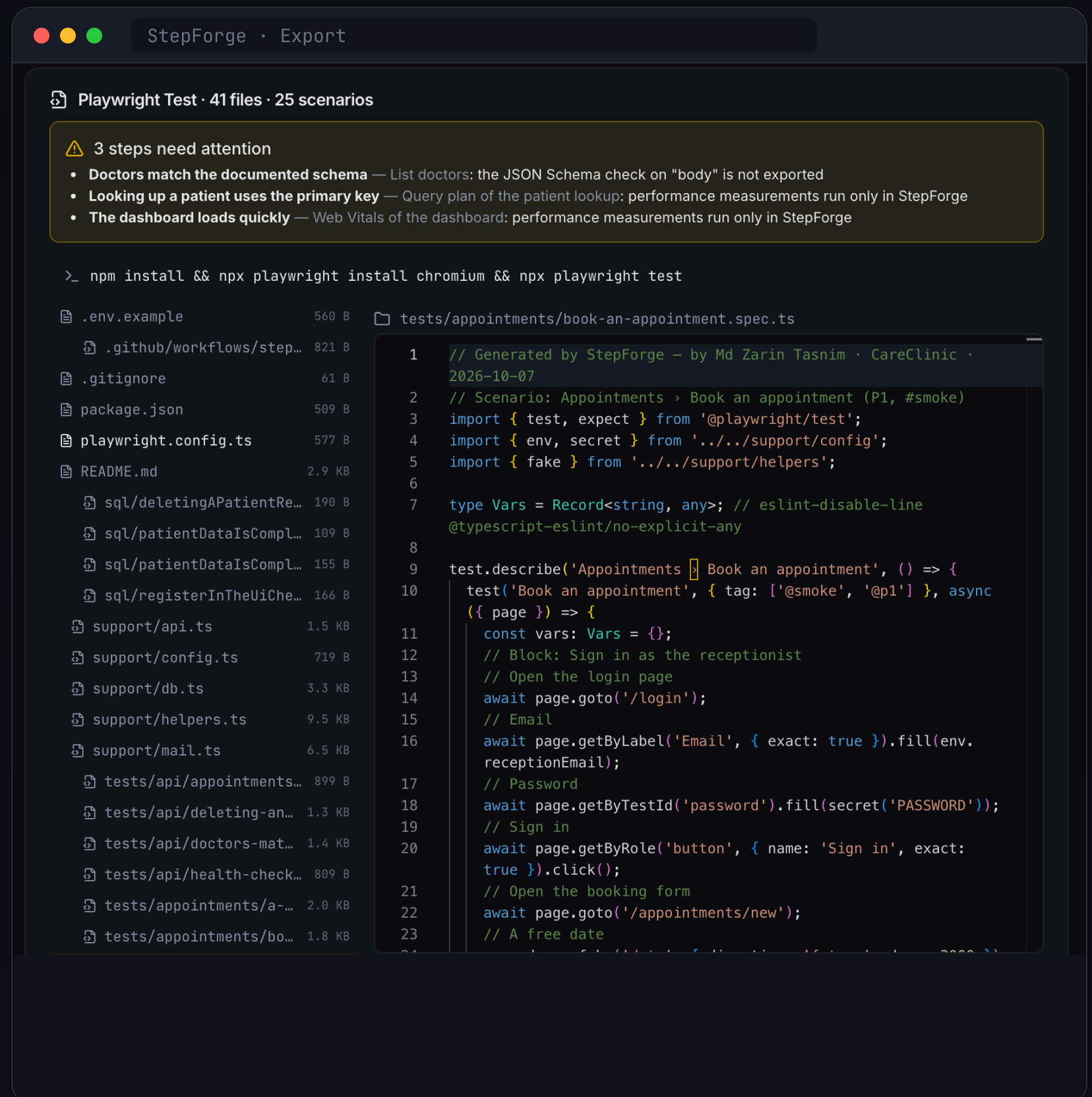
cURL

Markdown test cases

Gherkin

Excel test cases

Secrets become environment variables, CI files are included, and the exported Playwright project is run against the demo app in CI.

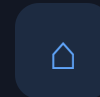


Built like a product, not a demo.



Secrets stay secret

AES-256-GCM at rest, never logged or exported, masked as •••• in the UI, reports and generated code.



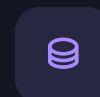
Local-first

Binds to 127.0.0.1 only, with a per-session token. One SQLite file. Optional local AI through Ollama — never a cloud API.



Explainable diagnosis

Failures are classified by readable YAML rules over captured facts, with unit tests for the rules.



Safe database access

Read-only and rollback by default. Production environments get a red banner and typed confirmation.



Self-healing locators

Each step keeps ranked locators — test id, role, label, text, CSS — and suggests a one-click fix when the primary one drifts.



Honest benchmark

Planted bugs are listed in a file StepForge never reads; a guard test fails the build if any app code mentions it.

RESULTS

Two demo apps. 25 planted bugs. Here's what StepForge caught.

25/25

planted bugs detected (100%)

24/25

diagnosed correctly (96%)

0

false alarms

83.6s

51 test runs, 45 scenarios

DEMO APP	SCENARIOS	TEST RUNS	PLANTED BUGS	DETECTED	DIAGNOSED CORRECTLY
CareClinic · clinic	25	29	13	13	12
ShopDesk · shop	20	22	12	12	12

6/6

UI bugs caught

6/6

API bugs caught

4/4

database bugs caught

1/1

email bugs caught

4/4

business-logic bugs caught

4/4

performance bugs caught

Bugs span API, database, UI, email, business logic and performance. Measured by scripts/benchmark.ts on an Apple M3 Pro; bugs are scored afterwards against a manifest StepForge never reads.

Modern, boring, free.

Core

Node.js 22+

TypeScript

npm workspaces

Zod

Server & data

Fastify

SQLite

Drizzle ORM

WebSockets

Dashboard

React

Vite

Tailwind CSS

Monaco

React Flow

TanStack

Testing engines

Playwright

autocannon

k6

Lighthouse

Mailpit

Databases

SQLite

PostgreSQL

MySQL

SQL Server

MongoDB

Quality

Vitest

Playwright E2E

ESLint

Prettier

GitHub Actions



BUILT BY

Md Zarin Tasnim

Need test automation or QA tooling? **Let's talk.**

Playwright · Cypress · Selenium · API & database testing · CI pipelines · custom QA tools

Hire me on Upwork → [upwork.com/freelancers/~01b847509724f9e1ff](https://www.upwork.com/freelancers/~01b847509724f9e1ff)

Source: github.com/zarindex/stepforge